DSA2102 Numerical Computation

Nawwaf Sudi

2026

Contents

1 Computer Arithmetics **2**

2 System of Linear Equation **4**

3 Least Square Problems **6**

4 Eigenvalue Problems **8**

5 Interpolation **13**

6 Numerical Differentiation **15**

1 Computer Arithmetics

This section concerns itself mostly with how a computer does arithmetics and the considerations of a computation problem. We'll cover the concept of precision and accuracy first. Consider this:

Precision vs Accuracy

Accuracy of the output of the calculation is the measure of how far it is from the true value. The *precision* of a calculation refers to how many significant digits (significand) that the output has, regardless of it's accuracy.

Let's talk a little bit about the problems we are trying to solve. A problem is refered to as well-posed if a solution exist, unique and varies continuously with the problem data. Do note that this property is seperate from our algorithm and although we would like our problems to be well-posed, some problems aren't.

Let's talk a bit more on the computation itself. In general, we would like a computation to not deviate far from the true value. Of course, having it be zero is largely unrealistic in parctice, since in order fo that to happen, we would like to already know the true value, and as such have no need for calculation.

This deviation from the true value, which we would call the error, largely derive from two main sources: The error that arises from the data itself, and the error that arises from our computation.

Take as an example, the computation that one would do when calculating the surface area of earth. To simplify the computation, we model the earth as a perfect sphere, and as such

$$A = 4\pi r^2$$

applies. Suppose that we are given the value of $\hat{r}$ as the radius of the earth. Then, we might arive a some value value $\hat{A}$. However, this value might deviate from the true value for a number of reasons:

1. Inaccuracy in the computation due to rounding and truncation.
2. Inaccuracy in the given data ($\hat{r}$)

The first one is cinsidered computational error, while the second belongs to data error. Since we are more interested in the factor we can control, we would be focusing more on the first one.

When talking about error, we refer to the absolute difference between our solution and the true value as the forward error. Similarly, we refer tio the absolute difference in the input is refered to as a backward error.

Generally, when designing an algorithm to solve a problem, we would like our problems to be *stable*. As in, small pertrubation in the input would likewise cause small variance in the output. We quantify this using the condition number

$$\kappa = \left| \frac{\text{forward error}}{\text{backward error}} \right|.$$

We can also say that

$$|\text{forward error}| = \kappa \times |\text{backward error}|.$$

As in, the condition number is the multiplier of error. We generally wan this number to be as low as possible, though in most cases, the minimum is one.

Do note that the condition number is a property of your algorithm, not hte problem itself. A problem is said to be well conditioned if the condition number is small. Similarly a problem is called ill-conditioned if the condition number is large.

Floating Point Arithmetics

At it's core, computer is a bunch of ones and zeros. To oversimplify a bit, the process involved in computations requires a way to represents numbers in a format that is precise, compact and minimizes errors. The standard way of doing so is through a floating point system. Consider the following sequence of characters in 1's and 0's:

$$0.00000000.00000000000000000000000.$$

The above is a sequence of numbers in base-2 of length 32. A (single) precision floating point system represents number in the above format. While there are other format, this chapter would only cover this format, and also the half precision floating point, which consists of 16 digits instead of 32.

The first digit, represents the sign of the number, with 0 being positive and 1 being negative. The second set of digits, which consists of 8 digits, determines the exponent of the digit. The third represents the significand of the number.

2 System of Linear Equation

This chapter primarily consists of the application of matrices in solving system of linear equations. Since a lot of this chapter is just review of linear algebra, I 'll omit a lot of it, and focus on the application instead.

Matrix Operations

In practice, matrix is simply a 2-dimension array, and operations on and between matrices can be summed up as iterating over the array. Let's talk addition first, which is performed on matrices of the same size. To compute it, we simply need to iterate on each entries of the matrix and add a corresponding entry on the other matrix. For a matrix A of size $m \times n$, we would need to perform $m * n$ operations.

Suppose matrices $A_{m\times p}$ and $B_{p\times n}$. A multiplication between them would involve, for each entry in the new matrix of size $m \times n$, an initialization followed by $2p-2$ operations (addition and multiplication). Thus, we have a total of $nm(2p-1)$ total operations.

However, we can further optimize this for matrices with special structures. Consider an upper triangular matrix $B_{n\times n}$ and some matrix $A_{m\times n}$. We can further minimize the operations into $mn\frac{n+1}{2}$ additions, and $mn\frac{(n-1)}{2}$ multiplications.

Furthermore, for a multiplication between two lower triangular matrix, the total operations would be $n(n+1)(n+2)/6$ multiplication and $(n-1)n(n+1)/6$ additions.

Norms and Errors

There are a couple of norms that each can be used to measure the 'size' of a vector, and consequently the error of matrix operations. This course would cover the euclidiean norm (2-norm), the p-norm and the infinity norm.

Let's begin with the general case of p-norm. It is defiend by

$$\|x\|_p = \left(\sum_{i=1}^{n} |x_i|^p\right)^{\frac{1}{p}}.$$

This way, you can form the euclidian norm

$$\|x\|_2 = \left(\sum_{i=1}^{n} |x_i|^2\right)^{\frac{1}{2}}.$$

and the infinity norm, which is the limit as $p \to \infty$. It yields

$$\|x\|_\infty = \max_i |x_i|,$$

as in, it is defined by the largest entry. Generalizing it to matrices, we can defien the following:

$$\|A\|_1 = \text{max absolute column sum}$$

$$\|A\|_\infty = \text{max absolute row sum}$$

$$\|A\|_2 = \text{max singular value in } A.$$

Now that we have a way of quantifying the size of a matrix, we can calculate the condition number of a matrix. Recall the definition of the condition number:

$$\kappa = \frac{|\text{forward error}|}{|\text{backward error}|} = \frac{\left|\frac{y}{\Delta y}\right|}{\left|\frac{x}{\Delta x}\right|}.$$

Rewriting it in terms of the transformation $A(x) = b$,

$$\kappa = \frac{\|\Delta x\|\|b\|}{\|\Delta b\|\|x\|} = \frac{\|A^{-1}(\Delta b)\|\|A(x)\|}{\|\Delta b\|\|x\|} = \frac{\|A^{-1}\|\|\Delta b\|\|A\|\|x\|}{\|\Delta b\|\|x\|}.$$

$$\kappa \leq \|A^{-1}\|\|A\|.$$

LU Factorization

For an LU factorization, we have two main algorithms:

1. Gaussian Elimination.
2. Cholseky Algorithm

In general, Cholesky is the one that is preferred, as it is a bit more effecient and stable. Regardless, Gaussian Elimination is worth knowing.

Gaussian Elimination is the method most people do by hand. It involves forming an upper triangular matrix, while storing the row operations in a matrix form. The resulting upper triangualr matrix is U, whilst the inverse of the row operation matrix is L. Writing the algorithm for a Gaussian Elimination is also quite straightforward: simple use forward subtitution to form the upper triangular matrix, whilst accumulating the operations in a seperate L vector of the same size.

One might notice that when the pivot entry of a matrixis zero, or very close to zero, the algorithm might run into some problem. To remedy this, we turn to pivoting, as in swapping the rows of a matrix such that zero divisions is avoided. The standard way to do this is that, for every column of a matrix, we pick the row that contains the largest entry below the diagonal, and swap that row as the pivot row for that iteration. This adds extra complexity in the accumulation of L, since it now has to include pivoting matrices.

Cholesky Factorization

Cholesky Factorization can only be done on matrices that is positive definite and symmetric. It is defined as a matrix that has a nonnegative determinant on all it'sleading principal submatrices.

The idea of a Cholseky Factorization can be demostratedd as such: Suppose a symetric positive definite matrix $A_{3\times 3}$. It's LU facotrization is can be written as

$$A = \begin{pmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{pmatrix} \begin{pmatrix} l_{11} & l_{21} & l_{31} \\ 0 & l_{22} & l_{32} \\ 0 & 0 & l_{33} \end{pmatrix}.$$

We can compute each l_{ij} based on the entry of A. Specifically, we can write the following:

$$\begin{aligned} a_{11} &= l_{11}^2 \\ a_{21} &= l_{11}l_{21} \\ a_{31} &= l_{11}l_{31} \\ a_{22} &= l_{21}l_{22} \\ a_{32} &= l_{21}l_{31} + l_{22}l_{32} \\ a_{33} &= l_{31}^2 + l_{32}^2 + l_{33}^2. \end{aligned}$$

We'd solve for each value of l_{ij}, which can be done following the above sequence.

3 Least Square Problems

Introduction

Suppose that you have a system $Ax = b$, where A is a $n \times m$ matrix where $n > m$. In such a system, we usually say that the system is overdetermined, and a solution wouldn't exist. Often times when presented with a problem like this, such as fitting a line over a couple of data points (elaborated further in later chapters concering interpolation,) we would like to find some solution that best fit our data.

Let's take a look at our previous example $Ax = b$. Suppose for $\mathbb{R}^2$we write them out as such:

$$\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \\ a_{41} & a_{42} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{pmatrix}.$$

Here, we couldn't find an exact solution to the problem. But, we can come up with some solution that minimize the 'error'. Such a problem is called a least square problem.

To understand how we could solve such a problem, we'll analyze $Ax = b$ once more. If we think of A as as some plane in $\mathbb{R}^2$, we can think of the existance of a solution for $Ax = b$ as that b lies somewhere within the span of A. Likewise, if there is no solution, b lise somewhere outside the span of A. Notice, however, that we can find some vector y within the span of A ($A\hat{x} = y$) such that $b = y + e$ for some vector e taht satisfies $A^T e = 0$ (e is perpendicular to the plane of A). As such, we can write the following:

$$0 = A^T e = A^T(b - y) = A^T(b - A\hat{x}).$$

$$A^T b = A^T A\hat{x}.$$

We can interpret the solution as finding the vector $\hat{x}$ that minimizes e (as it so happens for $\mathbb{R}^2$, the minimum is just the projection of b unto the plane.)

Notice the aforementioned expression

$$A^T b = A^T A\hat{x}.$$

We can rearrange it into the following:

$$(A^T A)^{-1} A^T b = \hat{x}.$$

We define

$$A^+ = (A^T A)^{-1} A^T.$$

Where A^+ is the pseudoinverse of A. It can be shown that for any $n \times m$ matrix A, A has a pseudoinverse that is positive.

QR Factorization

A QR factorization is simply decomposing a vector into $A = QR$, where Q is a orthogonal matrix $Q = Q^T$ and R is an upper triangular matrix. We could interpret this as taking an orthonormal basis Q and constructing A using R. Generally, it is preffered over LU as it is more stable and does not require pivoting as LU does.

When solving a QR factorization, we have a couple of options:

1. Gramm-Schmidtt Algorithm
2. Householder Reflections
3. Givens Rotation
4. Singular Value Decomposition (SVD)

Overall, Gramm-Schmidtt is not preffered as it quite slow. We'll talk about the first three in this chapter, and SVD in the later chapters concerning eigenvalue problems.

Gramm-Schmidtt

The idea behind Gramm-Schmidtt is to construct a set of orthonormal vectors by considering each columns of the matrix A, and iteratively subtracting it by it's projections on each previous columns and normalizing it, while storing the dot product of each vectors in an upper triangular matrix.

As one might expect, when doing the calculation by relying on previous results of computing each row of Q, error would be magnified. As such, when calculating a QR decomposition using this method, we would design the algorithm to iterate by rows instead.

Rotations and Reflections

Let's start with Householder Relfections. In short, a Householder reflection is an algorithm that, for each vector column in a matrix, try to reflect it such that it makes it orthogonal, then storing the reflector in a upper triangular matrix.

Before going further, let's discuss as to what it means for a vector to be reflected. Suppose some vector $\hat{b} \in \mathbb{R}^2$ that we wish to reflect in such a way that it is in the span of e_x. To do so, the easiet way is to find some line $\hat{w}$ that bisects the angle that $\hat{x}$ makes with the x axis, and subtract two times the vector $\hat{v}$ that is perpindicular to that line from $\hat{b}$. So esentially, reflecting is subtracting two times the perpendicular element of our choosen reflector, from $\hat{b}$.

In a Householder Reflection, we wish to do a relfection on each vector column of the matrix such that the entries below the pivot is zero'd out. To compute such a reflection, we follow

$$H_i = I - \frac{\hat{v}\hat{v}^T}{\hat{v}^T\hat{v}}.$$

We'll then solve for $\hat{v}$ such that the expression

$$H\hat{a}_i = \alpha e_i s$$

is statisfied. We'll repeat for each collumn until we have R, and we set $Q = H_1 H_2 \dots H_n$.

Even though this method is generally faster, for sparse matrices, or those with special structure, a Givens Rotation might be easier to perform. Contrary to a Householder Reflection, which seeks to zero out every single entry under the pivot in one go, a Givens rotation instead calculates a raotaion matrix that would zero out each individual entries.

Rotating a vector is another way to achieve the same effect, as they both preserve the norm. Going back to the first example, a vector $\hat{b}$ can be moved unto the span of e_x by rotating it, instead of a reflection. To do so, we multiply the vector $\hat{b}$ by the rotation matrix

$$R(\theta) = \begin{pmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{pmatrix}.$$

To do construct the matrix, we'll need to compute the sine and cosine of the rotation. Suppose that $\hat{b} = (x, y)$. Then, we have

$$\sin\theta = \frac{y}{\sqrt{x^2+y^2}}, \quad \cos\theta = \frac{x}{\sqrt{x^2+y^2}}.$$

For larger vectors, a intuitive way we can think of it is as 'adjusting' a line such that it lies only in one axis, and doing so by moving it one axis at a time.

In dering the QR decomposition, we do this rotation for each entry below the pivot, until we arrive at R, whilst accumulating the Q

4 Eigenvalue Problems

Recap

To recap, for a square matrix A and a non-zero vector x:

$$\begin{aligned} Ax &= \lambda x \\ 0 &= (A - \lambda I)x \\ p(\lambda) &= \det(A - \lambda I) = 0. \end{aligned}$$

The solution to x is called the Eigenvectors, and solution to λ are the Eigenvalues. $p(\lambda)$ is called the characteristic polynomial. $A - \lambda I$ is singular. This leads to the spectral theorem for symetric real matrices, which states that

Spectral Theorem for Real Matrices

A Symetric real matrix A can be diagonalized using an orthonormal basis of vectors.

$$A = QDQ^T.$$

A is said to be orthogonally diagonalizable.

This means that for any symetric real matrix, there always exists an orthonormal basis (Q) such that A is a simple diagonal matrix. Intuitively, this is changing the basis for A, such that A becomes simple scaling.

Polynomials and Eigenvalues

In numerical computation, the bulk of finding the root of a polynomial can be computed by computing the Eigenvalues, and such problem is classified as an eigenvalue problem. We do this by computing the eigenvalues using teh companion matrix, which is just the matrix form of the (monic, ie. the coeffecient for the largest term is 1) characteristic polynomial, given by

$$C_n = \begin{pmatrix} 0 & 0 & \cdots & 0 & -c_0 \\ 1 & 0 & \cdots & 0 & -c_1 \\ 0 & 1 & \cdots & 0 & -c_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & -c_{n-1} \end{pmatrix}.$$

For small polynomial ($n \leq 4$), we can just use the readily available formula to compute the root. The Abel-Ruffini Theorem states that there is no general formula for finding the root of polynomial of power ≥ 5 (TODO: Read), which is very common, and we would need to rely on algorithm to solve it iteratively, and thus find the root of the polynomial indirectly. Generally, such algorithm solves the root one at a time, so calculating all the root at once usually boils down to solving the eigenvalues.

Before doing so, let's take a look at the question of conditioning, uniqueness and existence of eigenvectors and eigenvalues in the real field. In general,

1. For an upper triangular matrix, the diagonal entries is the eigenvalues.
2. All symetric matrices have atleast one eigenvector and eigenvalues.
3. Every scalar multiple of an eigenvector is an eigenvector.
4. Every eigenvalues of a matrix is unique.
5. The algebraic multiplicity of a matrix A is $det(A - \lambda I)$.
6. The geometric multiplicity $dim(A - \lambda I)$ is equal to the number of unique eigenvalues of A. It is equal or less than the algebraic multiplicity. If it is strictly less, the matrix A is defective. Non-defective matrices of size $n \times n$ (ie. those with n linearly independent eigenvectors) are diagonalizable.

$$AX = XD \iff X^{-1}AX = D \iff A = XDX^{-1},$$

where X is a matrix of eigenvectors, and D is a diagonal matrix of the eigenvalues. If all the collumns in X is orthogonal, then we say that A is orthonormaly diagonalizable, and by the spectral theorem, $A = A^T$. More often than not, such matrices are symetric.

Operations on a matrix changes the eigenvalues. In general, the following applies:

1. Shifts $Ax = \lambda x \to (A - \sigma I)x = (\lambda - \sigma)x$
2. Inverses $Ax = \lambda x \to A^{-1}x = \frac{1}{\lambda}x$
3. Powers $Ax = \lambda x \to A^k x = \lambda^k x$
4. Polynomials $Ax = \lambda x \to P(A)x = P(\lambda)x$
5. Cholseky, QR, Elemination does nto preserve the non-zero eigenvalues.

The condition number (2-norm) for a non-defective eigenvalue problem is $X^{-1}AX = D$ is given by the condition numebr (2-norm) of X.

Singular Value Decomposition

Singular Value Decomposition

For $A \in \mathbb{R}^{m \times n}$,

$$A = U\Sigma V^T,$$

where $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ are orthogonal. The matrices $A^T A$ and AA^T are real symetric and orthogonally diagonalizable. Σ is a diagonal matrix consisting of the singular value of A (the square root of AA^T and $A^T A$ non-zero eigenvalues.)

When decomposing a matrix with SVD, we typically try to solve either $A^T A$ or AA^T for their eigenvalues and eigenvectors. Typically, we choose whichever is least complex to compute.

After calculating the eigenvalues and eigenvector, and forming either U or V, we would need to form the other base. To see this, recall that $A^T A$ is positive semi-definite, and can be written as

$$A^T A = VDV^T = V(\Sigma^T \Sigma)V^T.$$

For each column of V we have $A^T A v_i = \sigma_i^2 v_i$. Multiplying both side with A yields

Rootfinding Algorithm

Power Iteration and Variants

The idea behind power iteration is quite simple: For a matrix A that we would like to compute the eigenvalues and eigenvectors, we start with a vector v_0 (random, most like,) and repeatedly applying Av_i iteratively.

If A is non-defective and the largest eigenvalue is strictly larger than the other ones, we can analyze the algorithm's convergence quite easily:

Tutorial 6

Question 1

Use the Intermediate Value theorem to find an interval of length one that contains a root of the equation,

(a) $x^3 = 9$

(b) $\ln x + x^2 = 3$

Guess an root exists at [2,3]

$$f(2) = 8 - 9 = -1$$
$$f(3) = 27 - 9 = 18$$
$$.$$

By IVT, a root exists within that interval. Do the same thing for the second part: We guess an interval of [1,2]

$$f(1) = \ln 1 + 1 - 3 = -2$$
$$f(2) = \ln 2 + 4 - 3 > 0.$$

By IVT, a root exists within that interval.

Question 2

Consider the function $f(x) = x^3 + x - 1$

1. Write down the formula for x_k in terms of x_{k-1} for the Newton iteration.
2. Starting with an initial guess of $x = -0.7$, perform two iteration of the Newton method

Question 3

Find the eigenvalue of the matrix

$$\begin{pmatrix} 1 & -1 \\ 2 & 4 \end{pmatrix}.$$

$$\det \begin{pmatrix} 1-\lambda & 1 \\ 2 & 4-\lambda \end{pmatrix} = 0$$
$$(1-\lambda)(4-\lambda) + 2 = 0$$
$$(\lambda - 3)(\lambda - 2) = 0.$$
$$\lambda \in \{2, 3\}.$$

Question 4

Give an example to show that the eigenvalues can be changed when a multiple of one row is subtracted from another. Why is a zero eigenvalue not changed by the steps of elimination?

Take the matrix A from Q4. Suppose that we perform $R_2 \to R_2 - 2R_1$ on the matrix. We would have

$$\begin{pmatrix} 1 & -1 \\ 0 & 6 \end{pmatrix}.$$

Should we calculate the eigenvalue, we would get $\lambda \in \{1, 6\}$, clearly different than the original result.

To explain the reasoning behind the zero eigenvalues not being affected, we'll go back to the definition of eigenvalues and eigenvectors. For a matrix A with an eigenvalue $\lambda = 0$, we have

$$\det(A - \lambda I) = 0.$$

For any $\lambda \neq 0$, an elimination would shift the determinant of A such that the value of λ would need to compensate. For a $\lambda = 0$ however, $\det(A)$ is already 0 (singular), and so any row operations would not change the determinants, and λ need not compensate.

Question 5

Compute $A^T A$ and AA^T and their eigenvalues and unit eigenvectors for

$$A = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix}.$$

Use your result to factor $A = U\Sigma V^T$

For $A^T A$, we have

$$A^T A = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 1 \end{pmatrix}$$

$$\det \begin{pmatrix} 1-\lambda & 1 & 0 \\ 1 & 2-\lambda & 1 \\ 0 & 1 & 1-\lambda \end{pmatrix} = (1-\lambda)^2(2-\lambda) - 2(1-\lambda) = 0.$$

$$\lambda \in \{0, 1, 3\}.$$

Calculating the unit eigenvectors, we have

$$v_1 = \frac{1}{\sqrt{6}} \begin{pmatrix} 1 \\ -2 \\ 1 \end{pmatrix}, v_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 0 \\ -1 \end{pmatrix} v_3 = \frac{1}{\sqrt{6}} \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix}.$$

$$V = \begin{pmatrix} \frac{1}{\sqrt{6}} & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{6}} \\ -\frac{2}{\sqrt{6}} & 0 & \frac{2}{\sqrt{6}} \\ \frac{1}{\sqrt{6}} & -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{6}} \end{pmatrix}.$$

Calculating U,

$$v_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix} v_3 = \frac{1}{\sqrt{3}} \frac{1}{\sqrt{6}} \begin{pmatrix} 3 \\ 3 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

$$U = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} & \frac{1}{\frac{1}{\sqrt{2}}} \end{pmatrix}.$$

Putting it all together, we have

$$A = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} & \frac{1}{\frac{1}{\sqrt{2}}} \end{pmatrix} \begin{pmatrix} \sqrt{3} & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} \frac{1}{\sqrt{6}} & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{6}} \\ -\frac{2}{\sqrt{6}} & 0 & \frac{2}{\sqrt{6}} \\ \frac{1}{\sqrt{6}} & -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{6}} \end{pmatrix}.$$

Question 6

Let A be a $m \times n$ matrix with real entries and consider teh symetric eigenvalue problem

$$\begin{pmatrix} 0 & A \\ A^T & 0 \end{pmatrix} \begin{pmatrix} u \\ v \end{pmatrix} = \lambda \begin{pmatrix} u \\ v \end{pmatrix}.$$

Show that if λ, u, v satisfy the relationship above and $|u| = 1 = \|v\|$, then $\|\lambda\|$ is a singular value of A with corresponding left and right singular vectors u and v, respectively.

Breakdown teh expression:

$$A^T u = \lambda v, \quad Av = \lambda u.$$

Then,

$$AA^T u = \lambda^2 u \quad A^T A v = \lambda^2 v,$$

thus showing that λ is a singular value of A.

Question 7

The Fibonnaci Sequence $F_1, F_2, \ldots$is defined by $F_1 = 1, F_2 = 1$, and $F_n = F_{n-1} + F_{n-2}$ for $n \geq 3$. Define A by $A(x, y)^T = (y, x + y)^T$

(a) Show that $A^n(0, 1)^T = (F_n, F_{n+1})^T$ for each positive integer n.

(b) Find the eigenvalues of A.

(c) Find the basis of $\mathbb{R}^2$ consisting of eigenvectors of A.

(d) Use part (c) to compute $A^n(0, 1)^T$. Conclude that

$$F_n = \frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n\right).$$

(e) Use part (d) to conclude that for each positivei nteger n, F_n is the integer closest to

$$\frac{1}{\sqrt{5}}\left(\frac{1+\sqrt{5}}{2}\right)^n.$$

Starting with the first part. We'll use induction to prove it such. The base case is provided by $n = 1$, and

$$A\begin{pmatrix}0\\1\end{pmatrix} = \begin{pmatrix}1\\1\end{pmatrix} = \begin{pmatrix}F_1\\F_2\end{pmatrix}.$$

For the inductive steps, let $n = m + 1$. Asumming that the proposition holds for $n = m$

$$\begin{aligned} A^{m+1}\begin{pmatrix}0\\1\end{pmatrix} &= A\begin{pmatrix}F_m\\F_{m+1}\end{pmatrix} \\ &= \begin{pmatrix}F_{m+1}\\F_m + F_{m+1}\end{pmatrix} \\ &= \begin{pmatrix}F_{m+1}\\F_{m+2}\end{pmatrix}. \end{aligned}$$

so the proposition holds. For the eigenvalues of A, suppose that

$$A\begin{pmatrix}x\\y\end{pmatrix} = \begin{pmatrix}\lambda_0 & 0\\0 & \lambda_1\end{pmatrix}\begin{pmatrix}x\\y\end{pmatrix}.$$

Then,

$$\begin{pmatrix}y\\x+y\end{pmatrix} = \begin{pmatrix}\lambda_0 & 0\\0 & \lambda_1\end{pmatrix}\begin{pmatrix}x\\y\end{pmatrix}.$$

$$\lambda^2 x - \lambda x - x = 0 \implies \lambda^2 - \lambda - 1 = 0.$$

Solving the equation would yield the eigenvalues. We find

$$\lambda = \frac{1 \pm \sqrt{5}}{2}.$$

We can then proceed to find the eigenvectors of A. From our previous calculation, we find

$$\begin{pmatrix}y\\x+y\end{pmatrix} = \begin{pmatrix}\lambda_0 & 0\\0 & \lambda_1\end{pmatrix}\begin{pmatrix}x\\y\end{pmatrix}.$$

5 Interpolation

We'll start out with polynomial interpolation, which is basically try to fit a polynomial such that it matches the given data points (we'll skip over trig and rational interpolations in this course.)

In general, for a polynomial of degree d, a solution exists when the number of the given data points is no more than $d+1$, and the solution is unique as long as there is exactly $d+1$ points. Computing the polynomial depends on the basis matrix. One can choose any basis, each with their own advantages and disadvatages. Here, we'll be covering the monomial, Langrange and Newton basis.

In general, a monomial basis is computaionally unstable due to ill conditioned matrices, but conceptually, they are quite simple. Likewise, the Langrange basis is easy to implement, but computationally expensive, and quite rigid, requiring recalculating the whole thing for any additional points. Newton is the preferred method, as well will see.

I'll start with the monomial basis matrix, also known as the Vandermonde matrix. It looks something like this:

$$X = \begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^d \\ 1 & x_1 & x_1^2 & \cdots & x_1^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^d \end{pmatrix} = \begin{pmatrix} M_0(x_0) & M_1(x_0) & M_2(x_0) & \cdots & M_d(x_0) \\ M_0(x_1) & M_1(x_1) & M_2(x_1) & \cdots & M_d(x_1) \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ M_0(x_n) & M_1(x_n) & M_2(x_n) & \cdots & M_d(x_n) \end{pmatrix}.$$

This looks like

$$\begin{aligned} p(x) &= a_0 + a_1 x^2 + \ldots + a_n x^d \\ &= X\overline{a}. \end{aligned}$$

Example Find the polynomial that fit the points $(-2,-27),(0,-1),(1,0)$.

$$\begin{pmatrix} 1 & x_0 & x_0^2 \\ 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \end{pmatrix} \begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix}.$$

$$\begin{pmatrix} 1 & -2 & 4 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix} = \begin{pmatrix} -27 \\ -1 \\ 0 \end{pmatrix}.$$

Let's consider another basis: the Langrange Basis. It is defined as follows: Suppose that you have some points $(x_1,x_1),\ldots,(x_n,y_n)$. For each of these points, we form the function

$$\ell_i(x) = \prod_{k=0,k\neq i}^{n} \frac{x-x_k}{x_i-x_k}.$$

Then, we form the polynomial

$$P(x) = \sum_{i=0}^{n} y_i \ell_i(x).$$

In matrix form, this is the equivalent of the system

$$P(x) = Ly^T.$$

$$P(x) = \begin{pmatrix} \ell_1 \\ \vdots \\ \ell_n \end{pmatrix} \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}^T.$$

Let's take a look at $\ell_i(x)$ first. We can see this as an indicator function, that takes on the value of 1 when $x = x_i$ and 0 otherwise. From here, we can see that $P(x)$ constructed from a langrange basis is simply the summation of these indicator functions terms scaled by the corresponding value of y_i. What you would arrive at is a continuous polynomial that returns the value of y_i at x_i.

Example Take the example for the Vandermonde matrix. We have

$$\begin{aligned}
\ell_1(x) &= \frac{x(x-1)}{6} \\
\ell_2(x) &= \frac{(x+2)(x-1)}{-2} \\
\ell_3(x) &= \frac{(x+2)x}{-1} \\
P(x) &= -\frac{27}{6}x(x-1) + \frac{1}{2}(x+2)(x-1).
\end{aligned}$$

Consider another basis: the Newton Basis.

Piecewise Interpolation

Orthogonal Polynomials

6 Numerical Differentiation